

Nguyên lý hoạt động của web crawler, bóc tách dữ liệu, tư duy phân tích trang và phát hiện rủi ro web

Tóm tắt điều hành

Một crawler hiện đại không chỉ là vòng lặp “tải HTML rồi parse”. Trong thực tế, nó là một pipeline gồm: lấy nội dung và tài nguyên theo lịch, tôn trọng tín hiệu điều tiết như `robots.txt`, `429 Too Many Requests`, `Retry-After`, rồi mới đến parsing, render, trích xuất, lưu trữ, và khử trùng lặp. Những framework phổ biến phản ánh rất rõ mô hình này: Scrapy có execution engine, scheduler, downloader, spider middleware và item pipeline; Crawl4AI bổ sung dispatcher, rate limiter, bắt sự kiện mạng/console và nhiều tham số phục vụ render động. ¹

Với trang web hiện đại, lớp dữ liệu thực tế thường nằm sâu hơn HTML ban đầu. Dữ liệu có thể đến từ DOM sau khi JavaScript chạy, từ `fetch()` hoặc `XMLHttpRequest`, từ `iframe`, từ shadow DOM, từ lazy-loading, hoặc thậm chí từ module WebAssembly. Vì vậy, phân tích đúng thường bắt đầu bằng câu hỏi: “Dữ liệu mình cần xuất hiện ở source tĩnh, trong JSON-LD, trong XHR/Fetch, hay chỉ sau khi UI thay đổi?” Playwright, Puppeteer, Selenium và Crawl4AI đều có API để đánh giá JS trong ngữ cảnh trang, chờ selector/condition, chuyển vào frame, hoặc chặn và quan sát request/response. ²

Khi “nhìn” một trang để quyết định chiến lược, tư duy đúng là ưu tiên các tín hiệu bền vững và có ngữ nghĩa: structured data như JSON-LD/Microdata/RDFa, mẫu URL, mẫu DOM lặp lại, nhãn semantic, API endpoint, và quan hệ giữa các entity. Cách tiếp cận này bền hơn việc dựa hoàn toàn vào CSS class ngẫu nhiên do bundler sinh ra. Nghiên cứu wrapper induction như RoadRunner và các công trình sau đó cho thấy so khớp cấu trúc những trang tương tự, tận dụng DOM/XPath, record lặp, và template-based regularities là một cơ sở rất mạnh cho IE trên web. ³

Về bảo mật, báo cáo này trình bày theo hướng **phòng thủ và kiểm thử được ủy quyền**. Phần “quét/phát hiện lỗ hổng” vì vậy tập trung vào discovery surface, fuzzing/variation có kiểm soát, DOM-based XSS, CSRF, IDOR/BOLA, vấn đề auth/session, CORS, lộ thông tin, và cách phối hợp scanner tự động với kiểm tra thủ công. Với nguồn chính thức, Burp nhấn mạnh scanner nên hỗ trợ manual testing; ZAP nói rõ active scan là một cuộc tấn công và không nên dùng trên ứng dụng bạn không sở hữu; OWASP WSTG cung cấp test objective cho từng nhóm lỗi quan trọng. ⁴

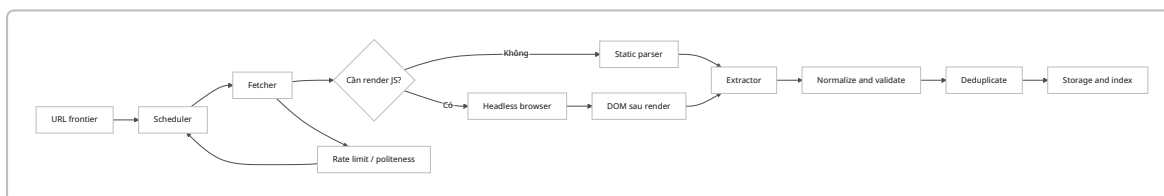
Phạm vi và giả định

Người dùng chưa nêu mục tiêu cụ thể, domain đích, mức ủy quyền, trạng thái đăng nhập, loại dữ liệu cần lấy, hay phạm vi quét bảo mật. Vì vậy, các mục sau đều được xem là **không xác định** và báo cáo **không giả định** thay người dùng: mục tiêu kiểm thử, tài khoản/phiên đăng nhập, môi trường production hay staging, biên độ tải chấp nhận được, cơ chế WAF/CDN, ràng buộc pháp lý/hợp đồng, và dữ liệu nhạy cảm có liên quan. Điều này đặc biệt quan trọng vì `robots.txt` chỉ là tín hiệu điều phối crawler chứ không phải cơ chế phân quyền truy cập; còn active scanning có thể bị xem như hành vi tấn công nếu không có ủy quyền rõ ràng. ⁵

Bởi vậy, phần crawler trong báo cáo được viết theo hướng **thu thập hợp pháp, minh bạch, có điều tiết**; còn phần bảo mật được viết theo hướng **kiểm thử phòng thủ trên tài sản do bạn sở hữu hoặc được phép kiểm tra**. Khi chưa có scope, cách làm an toàn nhất là đi từ thụ động đến chủ động: bắt đầu bằng review nội dung, cấu trúc mạng, header, tài nguyên tĩnh, structured data, và mapping bề mặt đầu vào; chỉ chuyển sang kiểm tra chủ động sau khi đã xác nhận phạm vi, cửa sổ thời gian, rate limit nội bộ và tiêu chí dừng. ⁶

Kiến trúc chung của crawler

Về mặt thực thi, kiến trúc “chuẩn” của crawler có thể hình dung như một chuỗi phối hợp giữa lấy request, lên lịch, fetch, render, parse, extract, store và dedupe. Trong Scrapy, execution engine điều phối data flow giữa spider, scheduler và downloader; scheduler giữ request trong cấu trúc dữ liệu persistent hoặc non-persistent rồi cấp lại theo thứ tự ưu tiên; item pipeline xử lý item tuần tự để validate, làm sạch, loại bản sao hoặc lưu cơ sở dữ liệu. Trong Crawl4AI, `arun_many()` dùng dispatcher để quản concurrency, bộ nhớ và rate limiting; `RateLimiter` có backoff cho `429/503`; và cấu hình crawl cho phép kiểm soát cache, JS execution, selector, robots, delay, iframe và shadow DOM flattening. ⁷



Sơ đồ trên là dạng rút gọn của pipeline mà tài liệu Scrapy và Crawl4AI mô tả: scheduler giữ frontier, fetcher tải nội dung, phần render chỉ kích hoạt khi trang phụ thuộc JS, sau đó extractor lấy record, pipeline chuẩn hóa và dedupe trước khi lưu. Phần “politeness” thực chất là một lớp điều tiết bao quanh fetch/schedule, dựa trên `robots.txt`, delay, concurrency slot và phản hồi máy chủ. ⁸

Thành phần	Vai trò kỹ thuật	Điểm cần chú ý
Fetching	Tải resource và response từ frontier; trong Scrapy luồng đi qua downloader middleware; trong Crawl4AI có thể chạy song song với dispatcher. ⁹	Phải tách HTML chính, tài nguyên phụ, và API/XHR để tránh “parse nhầm lờ”. ¹⁰
Parsing	Với HTML tĩnh, BeautifulSoup hỗ trợ CSS selector; lxml hỗ trợ XPath đầy đủ. ¹¹	Beautiful Soup tự ghi rõ nếu chỉ cần CSS selector thì parse bằng <code>lxml</code> sẽ nhanh hơn. ¹²
Rendering	Headless/browser automation chạy JS, chờ điều kiện, thao tác UI, truy cập frame/shadow DOM. ¹³	Dùng khi source tĩnh thiếu dữ liệu hoặc DOM thay đổi sau tương tác. ¹⁴
Politeness	Tôn trọng <code>robots.txt</code> ; Crawl4AI có <code>check_robots_txt</code> ; Scrapy có <code>ROBOTSTXT_OBEY</code> . ¹⁵	RFC 9309 nói rõ đây không phải authorization, nhưng là quy ước crawler nhân tôn trọng. ¹⁶

Thành phần	Vai trò kỹ thuật	Điểm cần chú ý
Rate limiting	<code>429 Too Many Requests</code> biểu thị vượt ngưỡng; phản hồi có thể kèm <code>Retry-After</code> ; Crawl4AI dispatcher dùng exponential backoff cho <code>429/503</code> ; Scrapy có <code>DOWNLOAD_DELAY</code> , randomize delay và AutoThrottle. ¹⁷	Delay ngẫu nhiên và auto-throttle giúp giảm burst và phản ứng theo độ trễ thực tế. ¹⁸
Scheduling	Scheduler quyết định thứ tự request và có thể dùng priority queue/slot theo domain. ¹⁹	Chất lượng frontier quyết định coverage, fairness và khả năng resume. ²⁰
Storage	Scrapy Item Pipeline xử lý làm sạch, validate, dedupe và lưu; Crawl4AI self-hosting nhấn mạnh kiểm soát hạ tầng và dữ liệu. ²¹	Nên tách raw HTML, cleaned HTML, extracted JSON và metadata crawl. ²²
Deduplication	Scrapy dùng request fingerprint để nhận diện cùng resource, mặc định bỏ qua fragment và thường không cần tính cookie/session header. ²³	Đây là điểm sống còn để tránh loop, lãng phí băng thông và kết quả trùng lặp. ²⁴

Một điểm thực chiến quan trọng là **đừng render nếu chưa cần**. Nếu dữ liệu đã có trong HTML, JSON-LD, hoặc script data dễ parse, parsing tĩnh bằng lxml/BeautifulSoup thường nhanh, nhẹ và ổn định hơn. Chỉ khi dữ liệu nằm sau tương tác, lazy-loading, shadow DOM, frame hoặc API ẩn sau JS mới nên chuyển sang Playwright/Puppeteer/Selenium/Crawl4AI render mode. Đây là một suy luận trực tiếp từ việc lxml được tài liệu BeautifulSoup mô tả là nhanh hơn cho CSS-style extraction, trong khi Playwright/Puppeteer/Crawl4AI đều dành nhiều API cho evaluation, frames, waits và request interception. ²⁵

Bóc tách dữ liệu và các lớp hiển thị

Lớp đầu tiên cần phân biệt là **HTML ban đầu** so với **DOM sau render**. `fetch()` và `XMLHttpRequest` đều cho phép trang lấy dữ liệu mà không refresh toàn bộ; vì thế việc “View Source” khác với “Elements” là bình thường. Selenium cũng nhấn mạnh phần tử trong SPA có thể được thêm hoặc đổi trạng thái hiển thị sau khi navigation hoàn tất; đó là lý do các waiting strategy tồn tại. Với Crawl4AI, `wait_for`, `js_code`, `scan_full_page`, `process_iframes` và `flatten_shadow_dom` đều là dấu hiệu cho thấy nhà công cụ xem đây là bài toán nhiều lớp chứ không phải chỉ parse HTML tĩnh. ²⁶

DOM parsing, CSS và XPath

Ở lớp tĩnh, CSS selector là cách nhanh và dễ bảo trì khi DOM có class/id hoặc cấu trúc semantic tương đối ổn định. BeautifulSoup hỗ trợ `.css.select()` cũng như `select()` và `select_one()`; lxml hỗ trợ `xpath()` đầy đủ và ghi rõ `.find*()` thường nhanh hơn XPath đầy đủ nếu không cần truy vấn quá chọn lọc. Khi template thay đổi ít và cây DOM “sạch”, đây là tầng extraction nên ưu tiên trước. ¹¹

```
from bs4 import BeautifulSoup

html = open("page.html", encoding="utf-8").read()
soup = BeautifulSoup(html, "lxml")

title = soup.select_one("main h1").get_text(strip=True)
```

```
prices = [el.get_text(" ", strip=True) for el in soup.select(".price")]
print(title, prices[:3])
```

```
from lxml import html

tree = html.fromstring(doc_html)
rows = tree.xpath("//article[contains(@class, 'product-card')]")
names = [r.xpath("//h2/text()")[0].strip() for r in rows]
print(names[:3])
```

Hai ví dụ trên bám theo khả năng chính thức của BeautifulSoup về CSS selector và của lxml về `xpath()`. Nếu chỉ cần CSS selector, tài liệu BeautifulSoup khuyên parse bằng `lxml` sẽ nhanh hơn.

11

Render bằng headless browser và thực thi JavaScript

Khi dữ liệu chỉ xuất hiện sau tương tác hoặc sau khi JS chạy, cần chuyển sang headless browser. Playwright cho phép `page.evaluate()` để chạy hàm trong ngữ cảnh trang và lấy kết quả về môi trường automation; Puppeteer có `page.evaluate()` và `waitForSelector()`; Crawl4AI cho `wait_for="css:..."` hoặc `wait_for="js:..."`, rồi chạy `js_code` sau khi điều kiện đạt, thậm chí auto-scroll với `scan_full_page=True`. ²⁷

```
from playwright.async_api import async_playwright

async def collect_jsonld(url: str):
    async with async_playwright() as p:
        browser = await p.chromium.launch()
        page = await browser.new_page()
        await page.goto(url)
        await page.wait_for_load_state("networkidle")
        data = await page.evaluate("""
            () => [...document.querySelectorAll('script[type="application/
            ld+json"]')]
                .map(s => s.textContent)
            """)
        await browser.close()
        return data
```

Đây là mẫu an toàn, thiên về quan sát và trích structured data, sử dụng đúng cơ chế `page.evaluate()` và chờ tải trang mà Playwright mô tả. ²⁸

Shadow DOM, iframes, lazy-loading, XHR/Fetch và mutation

Shadow DOM là vùng DOM có tính đóng gói. MDN mô tả nó như một cây DOM “ẩn” gắn vào host; Playwright nói locators mặc định xuyên được shadow DOM, trừ XPath và closed-mode shadow roots; Crawl4AI còn có `flatten_shadow_dom` để resolve slot và làm phẳng shadow DOM trước khi capture HTML. Với `iframe`, Playwright có `frameLocator()`, Selenium yêu cầu chuyển frame trước khi tương tác, còn Crawl4AI có `process_iframes=True` để inline nội dung trong extraction một trang.

Lazy-loading thì có thể nằm ở mức trình duyệt (`loading="lazy"` cho `iframe`) hoặc ở mức ứng dụng dựa trên scroll/navigation. ²⁹

`MutationObserver` là công cụ rất hữu ích khi cần biết **lúc nào** phần tử bị thêm, xóa hay đổi attribute; còn `fetch()` /XHR interception và Network panel giúp nhìn thấy endpoint thật, payload thật, resource type thật. Chrome DevTools Network có filter riêng cho Fetch/XHR, JS, Wasm, WS; đồng thời cho xem initiator, timing, headers và response. Crawl4AI có thể capture toàn bộ network requests và console messages; Puppeteer/Playwright đều có API interception/routing cho request. ³⁰

```
// Puppeteer: quan sát API calls một cách tối giản
await page.setRequestInterception(true);
page.on('request', req => req.continue());
page.on('response', async res => {
  if (res.url().includes('/api/')) console.log(res.status(), res.url());
});
await page.goto(url, { waitUntil: 'networkidle2' });
await page.waitForSelector('main');
```

Đoạn trên chỉ dùng interception để quan sát request/response, bám theo API chính thức của Puppeteer về `setRequestInterception()` và `waitForSelector()`. ³¹

```
# Selenium: chờ phần tử trong iframe sẵn sàng rồi mới thao tác
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

WebDriverWait(driver, 10).until(
    EC.frame_to_be_available_and_switch_to_it((By.CSS_SELECTOR, "iframe"))
)
```

Mẫu này phản ánh đúng guidance của Selenium về waits và frame switching cho các UI động. ³²

Nhận diện obfuscation và anti-scraping

Điểm phân biệt quan trọng là **minification** không đồng nghĩa với **mã hóa/obfuscation**. Bundle gọn, tên biến ngắn, file lớn một dòng chỉ cho thấy code đã được tối ưu đóng gói; nhưng khi analyst thấy runtime decode hoặc runtime code generation như `atob()`, `eval()`, `Function()`, gọi `WebAssembly.instantiate()`, hoặc luồng `crypto.subtle`, thì nên xem đó là dấu hiệu có thêm một lớp biến đổi dữ liệu hoặc dời logic sang nơi khó quan sát hơn. `atob()` giải mã Base64; `eval()` và `Function()` đều động parse/execute JavaScript string và MDN coi đây là injection sink/rủi ro XSS; `WebAssembly.instantiate()` tải và instantiate module nhị phân; `SubtleCrypto` cung cấp primitive mật mã mức thấp ngay trên client. ³³

Về mặt phân tích, cách đọc lớp obfuscation nên đi theo chuỗi: **nguồn dữ liệu ở đâu → hàm biến đổi nào tác động lên nó → điểm nào biến dữ liệu thành DOM/JSON/rendered text**. Nếu thấy Base64, trước hết theo dấu `atob()` / `btoa` hoặc data URL; nếu thấy chuỗi được build rồi ném vào `eval()` hay `new Function()`, ưu tiên tìm nơi chuỗi được assemble; nếu thấy mạng tải `.wasm` hoặc Network panel hiện loại `Wasm`, cần xem module đó có đang xử lý parse/crypto/anti-bot hay không; nếu thấy

`window.crypto.subtle`, giả định rằng một phần validate/encode/decode đang diễn ra client-side và không nên kết luận “dữ liệu an toàn” chỉ vì nhìn không đọc được trong source. ³⁴

Cũng cần phân biệt **che giấu hiển thị** với **che giấu dữ liệu**. Shadow DOM, `iframe`, element thêm động, `display:none`, attribute thay đổi do JS, hay lazy-loading đều có thể làm analyst tưởng dữ liệu “không tồn tại”, trong khi thực tế nó chỉ nằm ở lớp DOM khác hoặc xuất hiện muộn hơn.

`MutationObserver`, chờ condition phù hợp, và capture request/response sẽ giúp phân biệt nhanh hai tình huống này. ³⁵

Về anti-scraping, các nguồn ưu tiên ở đây xác nhận ba nhóm rất rõ ràng: **rate limiting**, **CAPTCHA**, và **fingerprinting**. RFC 6585 định nghĩa `429 Too Many Requests`; OWASP OAT-009 mô tả CAPTCHA như biện pháp phân biệt người với bot; còn MDN/W3C mô tả fingerprinting là việc website kết hợp đặc tính browser/OS để nhận diện thiết bị hay trình duyệt. Trên mục tiêu hợp pháp, các dấu hiệu này nên được hiểu như “tín hiệu cần giảm tải hoặc cần cơ chế truy cập phù hợp”, không phải lời mời để né tránh hoặc vượt rào. ³⁶

“Honeypot element” được yêu cầu trong đề bài, nhưng trong bộ nguồn ưu tiên đã tra cứu ở đây **không có tài liệu chuẩn đủ cụ thể** để mô tả một cách đáng tin cậy về biến thể triển khai. Vì vậy, báo cáo không suy đoán sâu về cơ chế này. Ở mức thực hành phân tích, nếu một phần tử/endpoint chỉ xuất hiện cho automation bất thường hoặc tạo tín hiệu telemetry kỳ lạ, hãy ghi nhận nó như một dấu hiệu anti-automation tùy biến và xác minh trong môi trường được ủy quyền.

Tư duy phân tích trang để xác định thông tin cần tìm

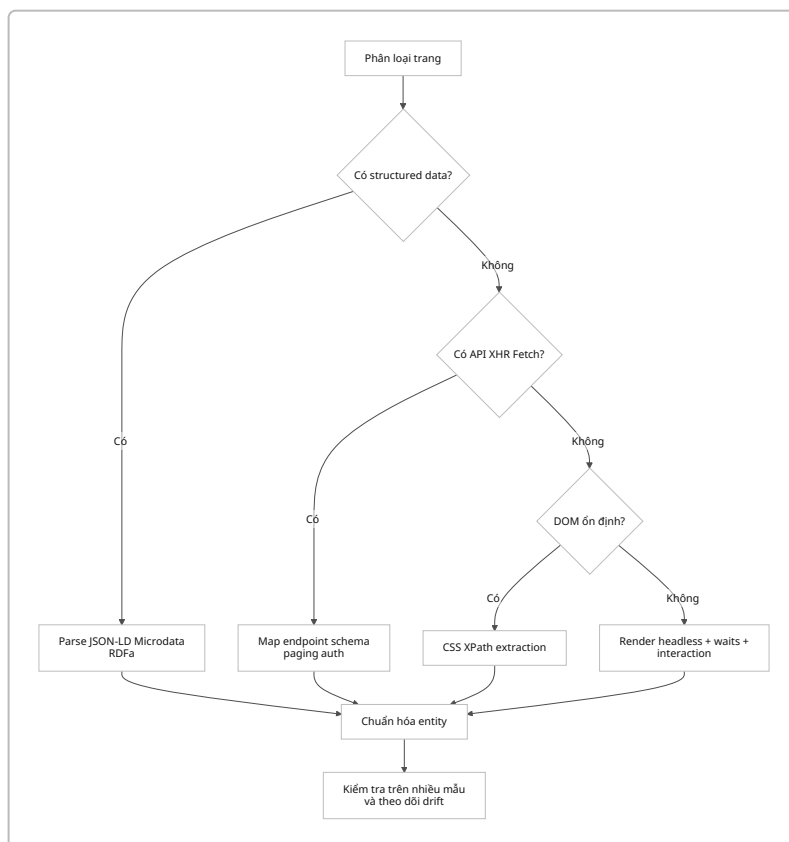
Câu hỏi trung tâm không phải “selector nào đúng”, mà là **“page này đang đại diện cho thực thể nào, và lớp nào đang giữ dữ liệu chuẩn nhất của thực thể đó?”**. Từ góc nhìn IE, các công trình kinh điển như RoadRunner cho thấy cách mạnh nhất là so sánh nhiều trang cùng loại để tìm phần cấu trúc ổn định và phần biến thiên; survey của Ferrara và cộng sự nhấn mạnh vai trò của DOM tree và XPath; các hướng hiện đại hơn còn đơn giản hóa DOM để tăng tính transfer và xem extraction như bài toán chọn tập XPath mô tả record semantically coherent. ³⁷

Một quy trình ra quyết định có hiệu quả thường đi như sau. Đầu tiên, xác định **loại trang**: trang danh sách, trang chi tiết, landing page, search result, form flow, hay SPA shell. Thứ hai, sẵn các **lỗi tắt có ngữ nghĩa**: JSON-LD/Microdata/RDFa, schema.org type như `Product`, `Article`, `Organization`, `Person`. Thứ ba, nếu không có structured data đáng tin, nhìn các **cụm DOM lặp** để suy ra record boundary. Thứ tư, mở Network và lọc Fetch/XHR/Wasm để xem endpoint nào mới là nguồn dữ liệu thật. Thứ năm, mới quyết định nên parse tĩnh hay render headless. ³⁸

Tín hiệu trên trang	Hàm ý	Chiến lược nên ưu tiên
Có <code>script[type="application/ld+json"]</code> hoặc Microdata/RDFa rõ ràng	Trang đã công bố entity ở dạng bán cấu trúc, thường ổn định hơn UI. ³⁹	Parse structured data trước, dùng DOM chỉ để bù trường thiếu.
Danh sách record lặp theo card/list/item	Đây là template-based extraction cổ điển; record boundary có thể suy được từ DOM lặp. ⁴⁰	Chọn selector/XPath theo record container trước, field sau.

Tín hiệu trên trang	Hàm ý	Chiến lược nên ưu tiên
DOM source nghèo nhưng Network có Fetch/XHR trả JSON	HTML chỉ là shell, API mới là source of truth. ⁴¹	Quan sát endpoint, schema response, pagination, auth, cache.
Dữ liệu xuất hiện sau click/scroll/tab switch	Trang phụ thuộc JS state transition. ⁴²	Dùng headless + waits + <code>js_code</code> /evaluate hoặc Selenium waits.
Web components/shadow DOM/iframe	Dữ liệu nằm ngoài light DOM chính. ⁴³	Dùng frame locator/switch frame, flatten shadow DOM nếu phù hợp.
URL theo pattern rõ, ví dụ <code>/product/</code> , <code>/article/</code> , <code>/api/...</code>	Có thể tách rule theo loại tài nguyên. ⁴⁴	Phân luồng crawler theo <code>url_matcher</code> , template hoặc extraction schema riêng.

Từ đây có thể rút ra một nguyên tắc rất thực dụng: **càng gần “nguồn dữ liệu chuẩn”, extraction càng bền**. Structured data thường bền hơn text rendered; API JSON thường bền hơn HTML trang trí; UI text sau render thường bền hơn pixel/visual OCR; còn chọn theo class hash ngẫu nhiên của CSS-in-JS thường là lựa chọn cuối cùng. Đây là suy luận được hậu thuẫn bởi tài liệu structured data và truyền thống web extraction theo template/DOM. ⁴⁵



Phát hiện lỗ hổng theo hướng phòng thủ

Với kiểm thử web bảo mật, quy trình an toàn nhất là đi từ **thụ động** → **bán chủ động** → **chủ động có kiểm soát**. OWASP WSTG bắt đầu bằng information gathering: review content để tìm lộ thông tin, liệt kê ứng dụng, review HTTP methods, rồi mới sang input validation, authorization, session management và client-side testing. Burp cũng khuyến nghị dùng Scanner để hỗ trợ manual testing, đặc biệt là quét trên request cụ thể thay vì scan tràn lan; còn ZAP nói thẳng active scanning là một cuộc tấn công và không nên dùng trên ứng dụng không thuộc quyền sở hữu của bạn. ⁴⁶

Discovery surface và parameter fuzzing

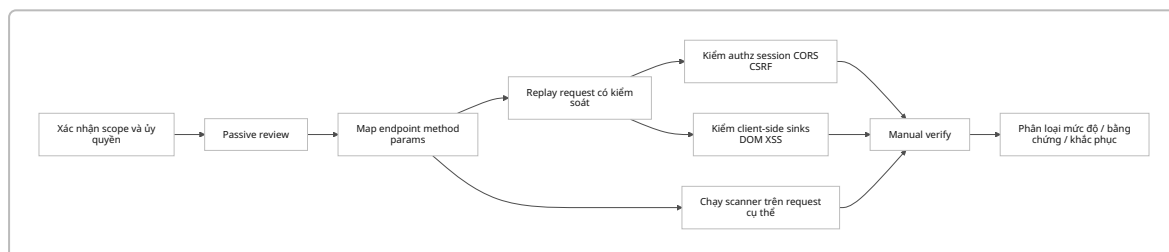
Bề mặt đầu vào không chỉ là form field. Nó gồm query string, POST body, JSON body, header, cookie, file upload, path segment, GraphQL variables, WebSocket message, trạng thái trong browser storage, và các request bị frontend “giữ hộ” trước khi gửi. WSTG có mục riêng về khả năng forge requests để bỏ qua GUI và gửi giá trị backend không mong đợi; Burp Repeater hỗ trợ gửi lại request thủ công với biến thiên tham số; Burp Intruder có chế độ Sniper để fuzz từng vị trí đầu vào riêng lẻ. Trong bối cảnh phòng thủ, mục tiêu của fuzzing là tìm điểm validate sai, lỗi exception, sai lệch mã trạng thái, hoặc behavior bất thường — không phải triển khai payload phá hoại. ⁴⁷

Những nhóm rủi ro chính cần kiểm

Nhóm rủi ro	Dấu hiệu nên tìm	Nguồn kiểm thử ưu tiên
DOM-based XSS	Dữ liệu từ URL, hash, <code>postMessage</code> , storage hoặc input đi vào DOM sink/client-side JS mà server không thấy được. OWASP lưu ý DOM XSS có thể xảy ra khi server không nhận biết được payload nào đã bị thực thi ở client. ⁴⁸	WSTG client-side testing; review nguồn/sink; manual validation trong browser.
CSRF	Hành động nhạy cảm phụ thuộc cookie/session nhưng thiếu CSRF token, SameSite phù hợp, hoặc kiểm tra origin/referrer thích hợp. OWASP định nghĩa đây là ép trình duyệt người dùng đang đăng nhập thực hiện hành động ngoài ý muốn. ⁴⁹	WSTG session management + CSRF cheat sheet.
IDOR/BOLA	Tham số tham chiếu trực tiếp object như <code>id</code> , <code>file</code> , <code>order</code> , <code>userId</code> ; thay đổi giá trị dẫn tới đọc/ghi object của người khác. OWASP mô tả rõ cần map mọi vị trí mà user input tham chiếu object rồi xác minh enforcement authorization. ⁵⁰	Authorization testing, API testing.
Auth/session issues	Exposed session variables, token yếu, scope/caching sai, session lộ qua kênh không mã hóa hoặc quản lý reuse sai. ⁵¹	Session management testing, auth testing, transport review.
Misconfigured CORS	<code>Access-Control-Allow-Origin</code> quá rộng, dùng credential sai, hoặc origin không tin cậy vẫn đọc được response. MDN mô tả rõ CORS là cơ chế header-based để browser cho phép cross-origin; OWASP có mục kiểm riêng cho CORS và cảnh báo tránh dùng <code>*</code> khi không cần. ⁵²	CORS testing bằng browser + header inspection + request replay.

Nhóm rủi ro	Dấu hiệu nên tìm	Nguồn kiểm thử ưu tiên
Info leakage	JS frontend lộ route ẩn, key công khai không nên công khai, stack trace, lỗi verbose, internal IP, admin path. WSTG có mục riêng review webpage content for information leakage; OWASP community nhấn mạnh error handling sai thường lộ chi tiết triển khai. ⁵³	Passive review source/assets, search code, error paths, HTTP responses.

Nếu cần một thứ tự thực hành ngắn gọn cho môi trường được ủy quyền, trình tự đáng tin nhất là: lập danh mục endpoint và state transitions; chụp baseline request/response cho từng flow; thay đổi từng tham số một để quan sát validation/authorization; đối chiếu state client-side với backend; cuối cùng mới chạy scanner tự động trên request cụ thể để tăng coverage. Cách làm này phù hợp với triết lý của Burp là scanner nên hỗ trợ manual workflow, và cũng phù hợp với WSTG khi tách rõ information gathering, forging requests, authz, session và client-side testing. ⁵⁴



Công cụ, bảng so sánh, bước tiếp theo và giới hạn

Công cụ và ví dụ ngắn

Công cụ	Khi nên dùng	Gợi ý sử dụng an toàn
Playwright	Site động, SPA, cần evaluate JS, thao tác frame/shadow DOM, route request. ⁵⁵	Dùng để quan sát DOM sau render và map network trước khi viết extractor chính thức.
Puppeteer	Tương tự Playwright, đặc biệt tiện cho interception và thao tác Chrome-family. ⁵⁶	Phù hợp khi cần quan sát request/response hoặc lấy DOM sau render.
Beautiful Soup / lxml	HTML tĩnh, extraction nhanh, rule-based. ¹¹	Ưu tiên khi dữ liệu đã nằm sẵn trong source hoặc structured data.
Selenium	Browser automation gần hành vi người dùng; hữu ích cho trang khó tương tác, iframe, wait state. ⁵⁷	Cần wait đúng và chuyển frame đúng; không nên lạm dụng sleep.
Scrapy	Crawl quy mô lớn, scheduling, dedupe, pipeline, AutoThrottle. ⁵⁸	Dùng cho frontier lớn; kết hợp parser tĩnh hoặc headless theo tầng.
Crawl4AI	Cần browser-based crawl có config rõ cho JS, iframes, shadow DOM, robots, network capture. ⁵⁹	Rất hợp cho extraction pipeline hiện đại và kiểm thử dữ liệu nhiều lớp.

Công cụ	Khi nên dùng	Gợi ý sử dụng an toàn
Burp Suite	Manual testing, replay, vary request, quét request cụ thể. ⁶⁰	Scanner nên đi sau mapping/manual review; verify issue bằng Repeater.
OWASP ZAP	Passive scan, baseline scan, active scan có scope. ⁶¹	Active scan chỉ chạy trên tài sản bạn sở hữu/được ủy quyền.
sqlmap	Công cụ PT tự động cho SQLi, rất mạnh và phải dùng cực kỳ cẩn trọng. ⁶²	Chỉ dùng trong lab cục bộ hoặc mục tiêu được ủy quyền rõ ràng.

```
# Scrapy settings tối thiểu cho crawl "lịch sử"
ROBOTSTXT_OBEY = True
AUTOTHROTTLE_ENABLED = True
DOWNLOAD_DELAY = 1.0
CONCURRENT_REQUESTS_PER_DOMAIN = 4
```

Cấu hình này bám thẳng vào hướng dẫn của Scrapy về `ROBOTSTXT_OBEY`, delay và AutoThrottle để giảm burst và điều tiết theo tải. ¹⁸

```
# ZAP baseline scan theo hướng thụ động trên staging/owned target
docker run -t ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py -t https://staging.example.com -r zap-report.html
```

ZAP có chế độ baseline/passive rất phù hợp cho vòng kiểm đầu tiên; còn active scan được tài liệu chính thức gọi rõ là một cuộc tấn công. ⁶³

```
# sqlmap: chỉ nên dùng trên lab cục bộ/được ủy quyền rõ ràng
sqlmap -u "http://127.0.0.1:8080/app?id=1" --batch --risk=1 --level=1
```

Ví dụ này được cố ý giới hạn ở `127.0.0.1` để giữ bối cảnh lab cục bộ. sqlmap là công cụ PT rất mạnh, có thể đi xa hơn nhiều so với detection cơ bản, nên tuyệt đối không nên dùng ngoài phạm vi được phép. ⁶²

Bảng so sánh trade-off

Quyết định	Ưu điểm	Nhược điểm	Kết luận thực dụng
Static parsing	Nhanh, nhẹ, dễ scale; Beautiful Soup còn ghi rõ <code>lxml</code> nhanh hơn khi chỉ cần selector. ¹¹	Bỏ lỡ dữ liệu sau JS, frame, shadow DOM, lazy-load. ⁶⁴	Chọn mặc định trước, render chỉ khi evidence cho thấy cần.

Quyết định	Ưu điểm	Nhược điểm	Kết luận thực dụng
Headless/ browser rendering	Thấy DOM sau render, chặn request, đánh giá JS, thao tác frame/shadow/UI state. ⁶⁵	Chậm hơn, tốn RAM/CPU hơn, pipeline phức tạp hơn. Đây là suy luận nhất quán từ bản chất headless browser và tài liệu tool. ⁶⁶	Dùng cho SPA, anti-pattern dữ liệu ẩn, hoặc UI-driven flow.
CSS selector	Dễ đọc, nhanh viết, phù hợp DOM rõ ràng. ¹²	Kém linh hoạt hơn XPath khi cần namespace, trực quan hệ phức tạp hoặc query rất chọn lọc. ⁶⁷	Tốt cho extractor vận hành; XPath cho case phức tạp.
Headless “stealth”	Có thể giảm false block trong vài môi trường. Crawl4AI có <code>simulate_user</code> ; nhưng đây chỉ là tính năng tool, không phải khuyến nghị mặc định. ⁶⁸	Giảm tính minh bạch, tăng rủi ro chính sách/pháp lý; khó audit hơn. RFC 9309 và config <code>user_agent</code> / <code>check_robots_txt</code> ủng hộ cách crawl nhận diện rõ ràng hơn cho use case hợp pháp. ⁶⁹	Với thu thập hợp pháp, ưu tiên transparency hơn stealth.
Scanner tự động	Tăng coverage, nhanh với lỗi phổ biến, tốt cho baseline. Burp và ZAP đều mạnh ở đây. ⁷⁰	Dễ sinh false positive/false negative; bỏ sót business logic; có thể gây tải hoặc hành vi tấn công nếu active. ⁷¹	Luôn ghép với manual verification.

Khuyến nghị pháp lý và đạo đức

Với crawl, tối thiểu nên xác minh điều khoản sử dụng, xác định user agent rõ ràng, tôn trọng `robots.txt` và phản hồi `429/Retry-After`, đặt delay/concurrency hợp lý, và tách môi trường thử nghiệm khỏi production nếu có thể. Selenium cũng nhắc người dùng web scraping nên quen với ToS của website vì có site cấm hoặc chặn automation. ⁷²

Với kiểm thử lỗ hổng, active scan, fuzzing có chủ đích, và công cụ SQLi chỉ nên chạy trên tài sản bạn sở hữu hoặc có ủy quyền bằng văn bản. ZAP nêu rõ active scanning là một cuộc tấn công; SQLMap tự mô tả mình là công cụ PT để phát hiện và khai thác SQLi. Không có scope rõ ràng, lựa chọn an toàn là dừng ở passive review và request replay tối thiểu trên môi trường lab/staging do bạn kiểm soát. ⁷³

Bảng hành động tiếp theo

Hành động tiếp theo	Mục tiêu	Ưu tiên	Điều kiện đầu vào
Xác định phạm vi, quyền kiểm thử, môi trường, cửa sổ thời gian	Tránh giả định sai và tránh gây tải ngoài ý muốn	Rất cao	Hiện tại không xác định
Chạy review tự động trên 5–10 URL mẫu	Phân loại page type, tìm structured data, map nguồn dữ liệu thật	Rất cao	Chỉ cần danh sách URL mẫu

Hành động tiếp theo	Mục tiêu	Ưu tiên	Điều kiện đầu vào
Quyết định “static trước hay headless trước”	Tối ưu tốc độ và độ bền của extractor	Cao	Kết quả review thụ động
Thiết lập crawler lịch sự	<code>robots.txt</code> , delay, concurrency, retry/backoff theo <code>429/503</code>	Cao	Scope crawl hợp pháp
Chuẩn hóa schema/entity cần lấy	Tránh trích sai trường và tăng khả năng monitor drift	Cao	Danh sách field mục tiêu
Lập ma trận input surface cho security review	Query, body, header, cookie, path, file upload, API	Cao	Có request/response baseline
Chạy baseline passive scan rồi kiểm thử công các request quan trọng	Tăng coverage nhưng vẫn kiểm soát safety	Trung bình-cao	Scope bảo mật được ủy quyền
Chỉ sau đó mới cân nhắc active scan/fuzzing sâu trên staging/lab	Xác minh lỗ hổng và thu bằng chứng rõ ràng	Trung bình	Có ủy quyền và tiêu chí dừng rõ

Giới hạn còn mở

Báo cáo này cố ý **không** đi vào payload khai thác chi tiết, bypass anti-bot, hay quy trình tấn công thực chiến ngoài môi trường được phép. Ngoài ra, do mục tiêu cụ thể **không xác định**, báo cáo không thể chọn sẵn extraction schema, selector cụ thể, endpoint cụ thể, hay playbook lỗ hổng theo stack. Nếu cần triển khai tiếp, phần còn thiếu lớn nhất là: domain đích, loại trang, field cần lấy, trạng thái đăng nhập, và tài liệu ủy quyền kiểm thử.

1 7 8 9 58 <https://docs.scrapy.org/en/latest/topics/architecture.html>
<https://docs.scrapy.org/en/latest/topics/architecture.html>

2 13 27 28 65 <https://playwright.dev/docs/evaluating>
<https://playwright.dev/docs/evaluating>

3 37 40 <https://www.vldb.org/conf/2001/P109.pdf>
<https://www.vldb.org/conf/2001/P109.pdf>

4 54 60 70 <https://portswigger.net/burp/documentation/desktop/testing-workflow/scanner-manual-testing>
<https://portswigger.net/burp/documentation/desktop/testing-workflow/scanner-manual-testing>

5 15 16 69 72 <https://datatracker.ietf.org/doc/html/rfc9309>
<https://datatracker.ietf.org/doc/html/rfc9309>

6 https://owasp.org/www-project-web-security-testing-guide/stable/4-Web_Application_Security_Testing/02-Configuration_and_Deployment_Management_Testing/02-Test_Application_Platform_Configuration.html
https://owasp.org/www-project-web-security-testing-guide/stable/4-Web_Application_Security_Testing/02-Configuration_and_Deployment_Management_Testing/02-Test_Application_Platform_Configuration.html

10 59 <https://docs.crawl4ai.com/advanced/network-console-capture/>
<https://docs.crawl4ai.com/advanced/network-console-capture/>

11 12 25 <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
<https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

14 32 57 64 <https://www.selenium.dev/documentation/webdriver/waits/>
<https://www.selenium.dev/documentation/webdriver/waits/>

17 36 <https://datatracker.ietf.org/doc/html/rfc6585>
<https://datatracker.ietf.org/doc/html/rfc6585>

18 <https://docs.scrapy.org/en/latest/topics/settings.html>
<https://docs.scrapy.org/en/latest/topics/settings.html>

19 20 <https://docs.scrapy.org/en/latest/topics/scheduler.html>
<https://docs.scrapy.org/en/latest/topics/scheduler.html>

21 <https://docs.scrapy.org/en/latest/topics/item-pipeline.html>
<https://docs.scrapy.org/en/latest/topics/item-pipeline.html>

22 44 68 <https://docs.crawl4ai.com/api/parameters/>
<https://docs.crawl4ai.com/api/parameters/>

23 24 <https://docs.scrapy.org/en/latest/topics/request-response.html>
<https://docs.scrapy.org/en/latest/topics/request-response.html>

26 41 https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

29 43 https://developer.mozilla.org/en-US/docs/Web/API/Web_components/Using_shadow_DOM
https://developer.mozilla.org/en-US/docs/Web/API/Web_components/Using_shadow_DOM

30 35 <https://developer.mozilla.org/en-US/docs/Web/API/MutationObserver>
<https://developer.mozilla.org/en-US/docs/Web/API/MutationObserver>

31 56 <https://pptr.dev/api/puppeteer.page.setrequestinterception>
<https://pptr.dev/api/puppeteer.page.setrequestinterception>

33 34 <https://developer.mozilla.org/en-US/docs/Web/API/Window/atob>
<https://developer.mozilla.org/en-US/docs/Web/API/Window/atob>

38 39 45 <https://schema.org/>
<https://schema.org/>

42 <https://docs.crawl4ai.com/core/page-interaction/>
<https://docs.crawl4ai.com/core/page-interaction/>

46 <https://owasp.org/www-project-web-security-testing-guide/v42/>
<https://owasp.org/www-project-web-security-testing-guide/v42/>

47 https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/10-Business_Logic_Testing/02-Test_Ability_to_Forge_Requests
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/10-Business_Logic_Testing/02-Test_Ability_to_Forge_Requests

48 https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/11-Client-side_Testing/01-Testing_for_DOM-based_Cross_Site_Scripting
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/11-Client-side_Testing/01-Testing_for_DOM-based_Cross_Site_Scripting

- 49 https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/05-Testing_for_Cross_Site_Request_Forgery
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/05-Testing_for_Cross_Site_Request_Forgery
- 50 https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/05-Authorization_Testing/04-Testing_for_Insecure_Direct_Object_References
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/05-Authorization_Testing/04-Testing_for_Insecure_Direct_Object_References
- 51 https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/04-Testing_for_Exposed_Session_Variables
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/04-Testing_for_Exposed_Session_Variables
- 52 <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>
<https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>
- 53 https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/01-Information_Gathering/05-Review_Webpage_Content_for_Information_Leakage
https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/01-Information_Gathering/05-Review_Webpage_Content_for_Information_Leakage
- 55 <https://playwright.dev/docs/locators>
<https://playwright.dev/docs/locators>
- 61 63 <https://www.zaproxy.org/docs/desktop/addons/passive-scanner/>
<https://www.zaproxy.org/docs/desktop/addons/passive-scanner/>
- 62 <https://sqlmap.org/>
<https://sqlmap.org/>
- 66 <https://www.selenium.dev/documentation/webdriver/>
<https://www.selenium.dev/documentation/webdriver/>
- 67 <https://lxml.de/xpathxslt.html>
<https://lxml.de/xpathxslt.html>
- 71 73 <https://www.zaproxy.org/docs/desktop/start/features/ascan/>
<https://www.zaproxy.org/docs/desktop/start/features/ascan/>